SUMMARY: INTRODUCTION TO PYTHON-II

SESSION OVERVIEW:

In this session, the students will be able to:

- Understand the basic syntax of Python
- Understand what are variables and why variables are used.
- Understand how to declare and assign a value to the variable.
- Understand difference between variables, identifiers, and keywords
- Understand the different data types, how to get data type and type conversion.

KEY POINTS AND EXAMPLES:

Understand the basic syntax of Python:

1. print() function

The print() function is used to display output to the screen
The syntax to use print() function is as follows:

```
print(<Objects>)
```

- <Objects> means that it can be one or more comma-separated 'Objects' to be printed.
- <Objects> must be enclosed within parentheses

Example: If we want to print “Hello, World!” in our code, we will write it in the following way:-

```
print("Hello, World!")
```

and, we get the output as:

```
Hello, World!
```

Python executed the first line by calling the print() function.

Note:- The quotes that are on either side of Hello, World! were not printed to the screen because they are used to tell Python that they contain a text.

```
print("Hi", "Hello")
```

The output will be:

```
Hi Hello
```

There are various ways the print() function can be used:

1) **Using the end parameter:** While using the print() function, we can use certain parameters to modify the output. A parameter is a value which can be provided to a function to modify the behaviour of that function. One of the parameters of the print function is "end".

   By default, the print function ends with a new line. The example is shown below:

```
print("Hello")
print("World")
```

   The output will be:

```
Hello
```

```
World
```

   The end parameter in the print() function allows you to specify what should be printed at the end of the output.

```
print("Hello", end = " ")
print("World")
```

   The output will be:

```
Hello World
```

```
print("Python", end='-')
print("programming")
```

The output will be:

```
Python-programming
```

2) **Using the sep parameter:** Like the "end" parameter, one other parameter is "sep". This can be used when multiple outputs are printed in the same print() method. the sep parameter in the print() function allows you to specify the separator between multiple values that are passed to the print() function.

The example is shown below:

```
print("apple", "banana", "orange", sep=', ')
```

The Output will be:

```
apple, banana, orange
```

2. input() function

The input() function in Python is used to accept user input from the keyboard. It allows a program to pause and wait for the user to enter some text or value, which is then returned as a text. The input() function in Python by default reads user input from the keyboard as a string.

```
user_input = input("Enter your name: ")
print("Hello")
```

In this example:

- The input() function displays the prompt "Enter your name: " to the user.
- The program waits for the user to enter some text.
- Whatever the user types is stored in the user_input.
- The program then prints "Hello".

```
fruit = input("Enter a fruit: ") #apple
color = input("Enter a color: ") #red
print("The " + fruit + " is " + color + ".")
```

The output will be:

```
The apple is red.
```

3. Comments

Comments help us understand code better by explaining what it does.
They act like notes, reminding us or others about important details in the code.
Comments make it easier to update and improve the code later on.

1. Single-line comments: Single-linecomments begin with the # symbol, and Python will ignore them:

```
# This is a single-line comment
print("Hello, world!")
```

2. Multi-line Comments: Multi-line comments, also known as block comments, are enclosed within triple quotes (''' or """).

```
'''
This is a multi-line comment.
It can span multiple lines and is enclosed within
triple quotes.
This is useful for documenting larger sections of
code.
'''
```

4. Identation

Indentation in Python is used to define the structure of code blocks. Other languages, like JavaScript or Java, use curly braces {}.

Think of writing code like building a sandwich. In Python, indentation acts like the layers in a sandwich. Each layer (or indentation level) represents a different part of the sandwich (or

block of code). Without the layers, it's hard to see where one part of the sandwich (or code block) starts and another ends. Just like how layers keep your sandwich organized and tasty, indentation keeps your code organized and understandable.

```
x=5
# Check if x is greater than 3
if x > 3:
    # This line is indented
    print("x is greater than 3")
```

Understand What are Variables and why variables are used:

Variables in programming are like containers that hold data. They're used to store information that can be accessed and manipulated throughout the program..

Variables are super important in programming for a few big reasons.

- Storing Data: Variables are like containers that hold different types of information, such as numbers or words, so we can use them later.
- Manipulating Data: Once we put information in a variable, we can change it or do things with it, like adding numbers together or combining words.
- Reusability: Instead of typing out the same information over and over again, we can put it in a variable and use it multiple times in our program.
- Readability: Giving variables clear names makes our code easier to understand for ourselves and others. It's like labeling things in a messy room so we can find them later.

Understand how to declare and assign a value to the variable:

In Python, there's no separate command or statement specifically for declaring variables. we don't need to explicitly declare the data type of a variable before using it. Instead, variables are automatically created the moment you assign a value to them. For example:

```
x = 42
```

In the above example, x is the name of the variable, and 42 is the value assigned to it. This statement initializes x with the value 42. After this line of code is executed, you can use x to access the value 42 throughout your program.

```
print(x)
```

The output will be:

```
42
```

another example with a text:

```
my_name = "John"
print(my_name)
```

The output will be:

```
John
```

The basic rules for naming variables:

1. Variable names can contain letters (a-z, A-Z), digits (0-9), and underscores (_).

2. Variable names must start with a letter or an underscore. They cannot start with a digit or any special characters.

3. Variable names are case-sensitive, meaning uppercase and lowercase letters are considered different. For example, the variable names Temp and temp are different.

4. Python keywords (reserved words) cannot be used as variable names. For example
   print = "Hello"
   In this example, print is a Python keyword used for printing output to the console. When we try to use it as a variable name and assign the value "Hello" to it, we'll encounter a syntax error because print cannot be used as a variable name.

5. Variable names cannot contain spaces or special characters, except for underscores (_).

Legal Variable Name:

```
my_variable = 42
this_is_a_variable = "Hello"
number_1 = 10
```

Illegal Variable Name:

```
1_number = 42  # Cannot start with a digit
my-variable = "Hello"  # Cannot contain special
characters like hyphens
for = 10  # Cannot use Python keywords as variable
names
```

Variables are called variables because their values can be varied. And the last re-assigned values stays in variable. For example,

```
Var = 10
Var = "Hello"
print(Var)
```

The output will be:

```
 Hello
```

we can assign multiple values to multiple variables in a single line using a process called "unpacking".

```
# Assigning multiple values to multiple variables
x, y, z = 10, 20, 30
# Printing the values of the variables
print("x:", x)
print("y:", y)
```

```
print("z:", z)
```

In this example:
We have three variables x, y, and z.
We assign the values 10, 20, and 30 to these variables in a single line.
When the code is executed, each variable gets assigned its respective value.

The output will be:

```
x: 10
y: 20
z: 30
```

This technique is particularly useful when you have multiple values that you want to assign to multiple variables simultaneously.

We can assign a single value to multiple variables in a single line by repeating the variable names separated by commas, followed by the value you want to assign.

```
# Assigning one  value to multiple variables
x = y = z = "Hello"
# Printing the values of the variables
print("x:", x)
print("y:", y)
print("z:", z)
```

In this example:
We assign the value "Hello" to the variables x, y, and z in a single line.
All three variables will have the same string value "Hello".

```
x: Hello
y: Hello
z: Hello
```

<u>Understand difference between variables, identifiers, and keywords:</u>

Variables:

- Variables are names that are used to store values in memory.
- They can hold different types of data such as numbers, strings, or boolean values.
- Variables are created when they are assigned a value.

Identifiers:

- Identifiers are names given to various programming elements such as variables, functions, classes, etc. So, variables are a specific type of identifier.
- Identifiers can consist of letters (both uppercase and lowercase), digits, and underscores.
- However, they cannot start with a digit and any special characters and they cannot be the same as Python keywords.

Keywords:

- Keywords are reserved words in Python that have special meanings and purposes.
- They are predefined by the Python language and cannot be used as identifiers (such as variable names).
- Examples of keywords include if, else, for, while, def, class, return, True, False, None, etc.
- These keywords have specific functionalities and are an integral part of Python's syntax.

Understand the different data types, how to get data type and type conversion:

Data types define the nature of the data that a variable can hold.

A data type tells a computer what kind of information you're working with. It helps the computer understand how to handle that information. For example, a data type could be "number," "text," "true/false," or "list of items."

Some common data types include:

- **Numbers**
    - Integers(int): Whole numbers without any decimal point.
    - Floating Point Numbers(float): Real numbers with a decimal point.
    - Complex Numbers(complex): complex numbers with a real and imaginary part.

  Example:

```
x = 10 #int
y = 3.14 #float
z = 2 + 3j #complex
```

- **Strings(str):** Sequence of characters enclosed within single (' ') or double (" ") quotes. strings are "immutable" which means they cannot be changed after they are created

  Example:

```
name = "John"
```

To output a string using the print function we use quotes. But in some cases, there can be a conflict when the same quotes are used inside the string. An example is shown below:

```
print('She is Ana's sister')
```

If you write the above script and run, it will throw an error, because we are using the single quotes inside the string also.

One possible solution for this problem is to use double quotes for the string

```
print("She is Ana's sister")
```

But this can also cause a problem for a more elaborated case like

```
print("She is Ana's sister. Last night she said
"Ana helps me with my homework.".")
```

The above script will also produce an error because now we are using both single quote and double quotes inside the string.

Using the backslash (\): To overcome this, we can use the "\" (backslash) operator while using single quotes or double quotes inside the string.

```
print('She is Ana\'s sister')
```

```
print("She is Ana's sister. Last night she said
\"Ana helps me
with my homework.\".")
```

The backslash helps the print function to understand that the quotes are a part of the string.

We can also use triple quotes (”””) to print the output. This can also be used to print multiple lines.

```
print("""She is Ana's sister
She is very intelligent.
Last night she said to me "Ana helps me in doing my
homework.""")
```

We can use double quotes or single quotes inside the triple quotes without using the backslash.

Escape Characters:

1. newline character: Adding a newline character (\n) allows you to control where line breaks occur within your printed output.

```
# Using newline character to break lines
print("""She is Ana's sister.\nShe is very
intelligent.\nLast night she said to me "Ana helps
me in doing my\nhomework.""")
```

The output will be:

```
She is Ana's sister.
She is very intelligent.
Last night she said to me "Ana helps me in doing my
homework".
```

2. tab character: \t inserts a tab character, which typically moves the cursor to the next tab stop or column.
It's useful for aligning columns of text, such as when printing tabular data or formatting output neatly.

```
# Using \t for tab character
print("Name:\t\tJohn")
print("Age:\t\t25")
print("Occupation:\tEngineer")
```

The output will be:

```
Name:           John
Age:            25
Occupation:     Engineer
```

- **Boolean Values(bool):** Represents True or False values

  Booleans are commonly used for making decisions
  Booleans can have two possible values: True and False. These values are predefined keywords in Python.
  The bool() function allows you to evaluate any value, and give you True or False in return,
  Empty values such as an empty string "" or 0 will result in False when passed to the bool() function.
  Non-empty values such as "Hi" or 12 will result in True when passed to the bool() function.

  For example,

```
print(bool(""))       # Output: False
print(bool(0))        # Output: False
print(bool("Hello"))  # Output: True
```

- **List, Tuple,set and Dictionary (To be covered later in the course)**
  They are used to store multiple items in a single variable.

Getting Data Type: we can use the type() function in Python to determine the data type of a variable or value. For example

```
x = 10
print(type(x))  # Output: <class 'int'>
```

```
x = True
print(type(x)) # Output: <class 'bool'>
```

Type Conversion: Type conversion, also known as typecasting, is the process of converting a value from one data type to another.

Python provides built-in functions for type conversion, such as int(), float(), str(), bool(), etc.

For example,

```
x=5
y=float(x)
print(y) # Output: 5.0
print(type(y)) # Output: <class ‘float’>
```

```
x = 10
y = str(x)  # Convert integer to string
print(type(y))  # Output: <class 'str'>
```

```
age_str = input("Enter your age: ")  # Get user
input as string
age_int = int(age_str)  # Convert string to integer
print(type(age_int))  # Output: <class 'int'>
```

In this example, the input() function is used to prompt the user to enter their age, which is stored as a string. Then, the int() function is used to convert this string into an integer, which is stored in the variable age_int. Finally, the type() function is used to verify the data type of age_int, confirming that it is indeed an integer.

```
is_valid_bool = True
is_valid_int = int(is_valid_bool)
print(type(is_valid_int))  # Output: <class 'int'>
```